Real-Time Vehicle Detection & Traffic Analytics for Indian Urban Roads Using Deep Learning

Master Project Documentation: From Scratch to End

Author: Manav Shah
Repository: github.com/manavshah409/Deep_Learning_UHV
Dataset: IISc AIM Urban Vehicle Heterogeneity (UVH-26) Dataset (Majority Voting Annotations)
Primary Detector: E1 YOLOv8s (Input Resolution: 640×640, Best Checkpoint: Epoch 22)
Hardware & Environment: Apple Silicon (M-Series, MPS Backend), Python 3.12.14, PyTorch 2.14.0, Ultralytics 8.4.146

Table of Contents

1. Executive Summary & Project Overview

2. Problem Statement & Domain Challenges

3. Data Engineering & Integrity Governance

• 3.1 Source Dataset: UVH-26

• 3.2 Full Catalogue Audit vs. Local Frozen Subset

• 3.3 14-Class Taxonomy & Class Imbalance

• 3.4 Data Integrity, Quarantine & Manifest Security

4. Experimental Evolution: From Scratch to End

• Phase 1 (E0): Compact One-Stage Baseline (YOLOv8n)

• Phase 2 (E1): Model Capacity Scaling (YOLOv8s)

• Phase 2 (E2): Input Resolution Ablation Study (640 vs. 960)

• Stages A–D (E3): Two-Stage Detector Exploration (Faster R-CNN ResNet-50-FPN)

• Stage E: Matched Calibration Evaluation & Complementarity Analysis

• Stage F (E4): Model Ensembling & Fusion Feasibility Study

• Phase 3A & 3B: Video Traffic Analytics, Tracking & Line-Crossing Counting

5. Master Performance & Latency Matrix

6. System Architecture & Codebase Layout

7. Streamlit Interactive Research & Faculty Dashboard

8. Software Engineering Rigor & Reproducibility Guarantees

9. Step-by-Step Execution & Reproduction Guide

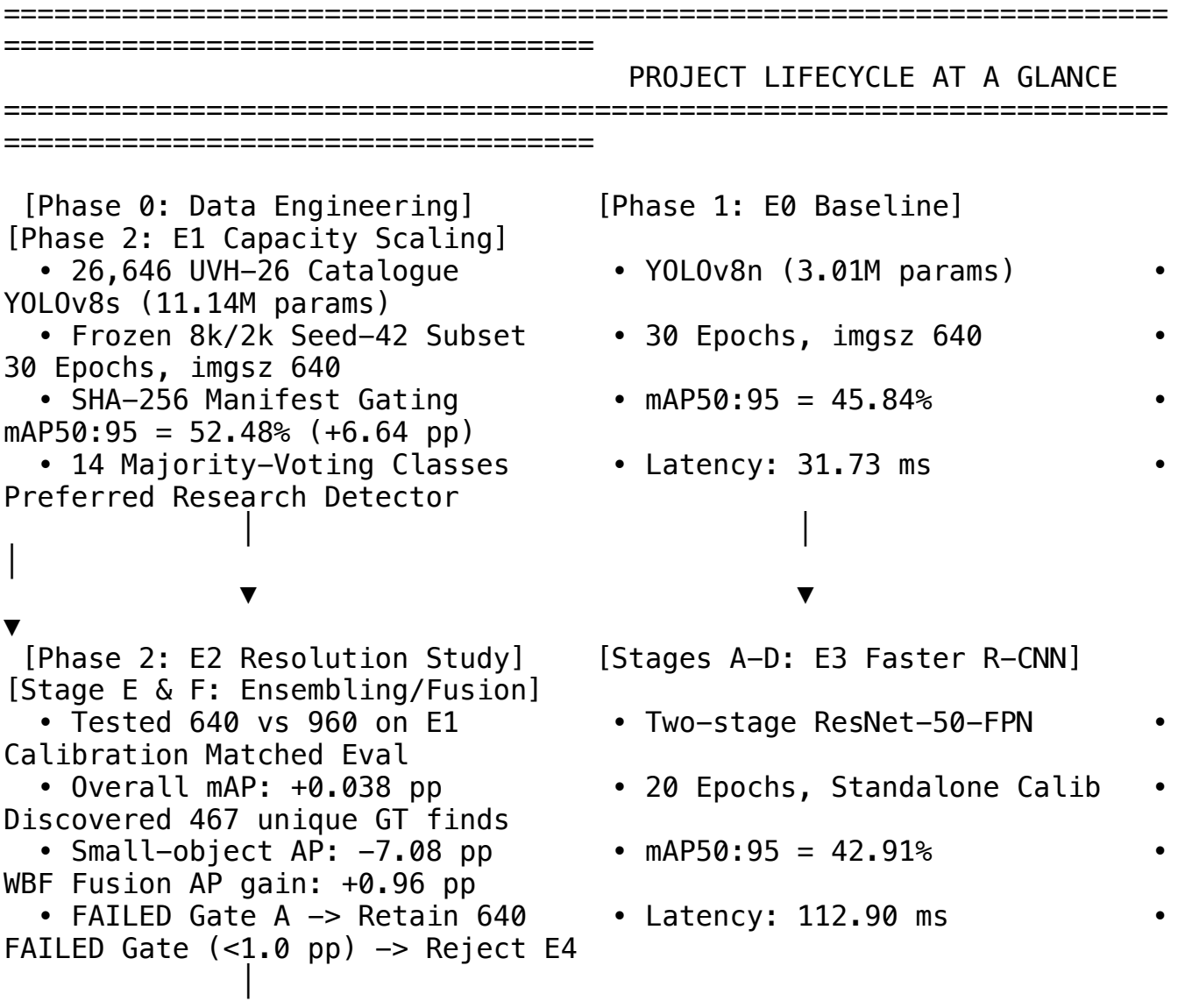
10. Failure Analysis, Limitations & Future Roadmap

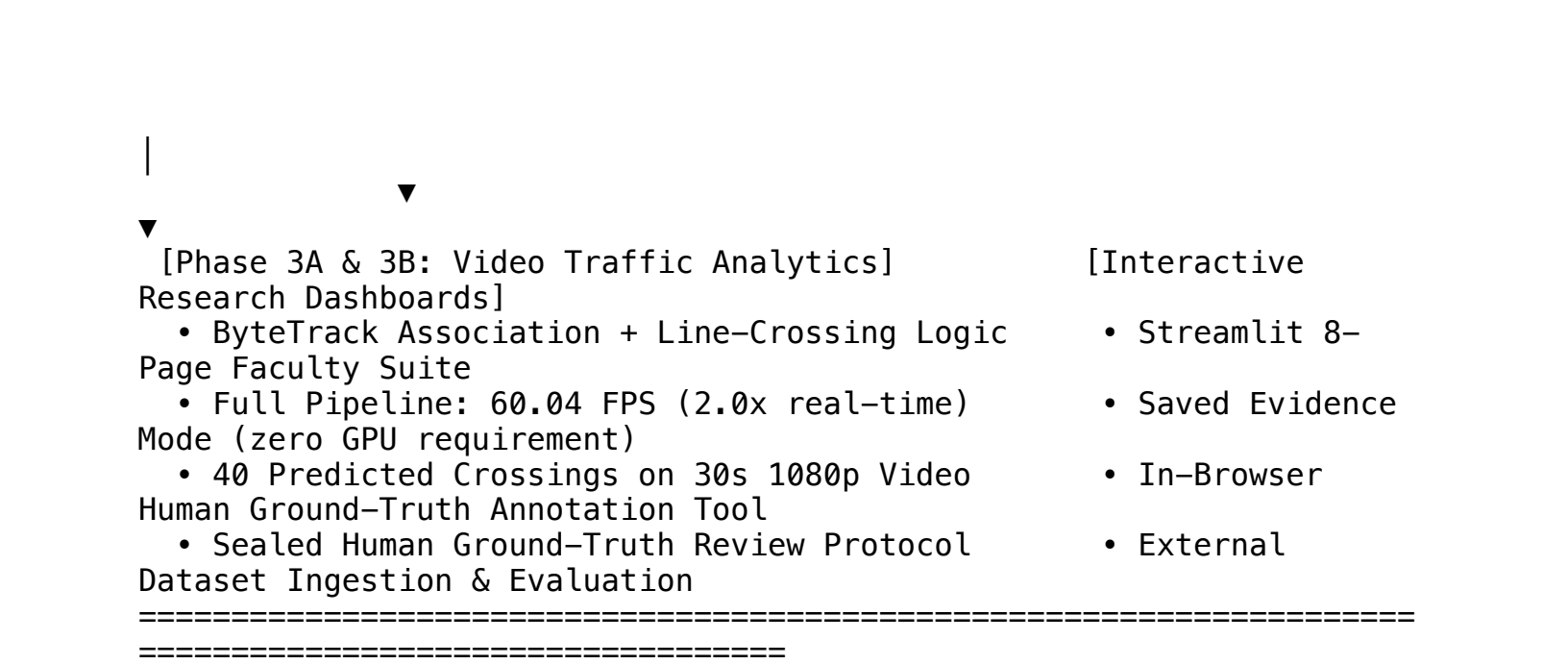
---

1. Executive Summary & Project Overview

This research project delivers a reproducible, scientifically audited deep learning pipeline for **real-time multi-class vehicle detection and traffic analytics on heterogeneous Indian urban roadways**.

Standard computer vision benchmarks (e.g., COCO, KITTI, Waymo) evaluate driving environments characterized by strict lane discipline, homogeneous car/truck categories, uniform vehicle scales, and moderate occlusion. In contrast, Indian urban traffic exhibits extreme density, non-lane-based vehicle flow, severe mutual occlusion, drastic scale disparities (from small motorized two-wheelers and bicycles to multi-axle commercial trucks), and unique vehicle categories such as auto-rickshaws (*Three-wheelers*), *Tempo-travellers*, and animal/hand carts (*Others*).





Key Milestones & Findings

1. **Optimal Detector Selection (E1 YOLOv8s):** YOLOv8s emerged as the preferred research detector, achieving **52.48% mAP@0.5:0.95** (an absolute gain of **+6.64 percentage points** over the YOLOv8n baseline at 45.84%), while maintaining real-time MPS latency of **30.29 ms** (33.01 still images/sec).

2. **Resolution Hypothesis Rejected (E2 960 Study):** An empirical ablation comparing 640 vs. 960 resolution revealed an overall mAP gain of only **+0.038 percentage points** and a **-7.08 percentage point decline** in small-object macro AP. Guided by pre-registered decision rules, training at 960 was formally rejected, saving compute.

3. **Two-Stage Complementarity vs. Fusion Tradeoff (E3 & E4):** Faster R-CNN achieved **42.91% mAP50:95** on calibration. Although Faster R-CNN detected **467 unique ground-truth objects** missed by YOLOv8s, Weighted Boxes Fusion (WBF) yielded only a **+0.96 pp gain** (falling short of the pre-registered 1.0 pp threshold) while tripling latency to **156.78 ms** (6.38 images/s). Fusion was formally rejected.

4. **Real-Time Video Analytics (Phase 3B):** An end-to-end multi-object tracking and directional line-crossing counting pipeline achieved **60.04 FPS on 1080p video** on Apple Silicon MPS (2.0× real-time), with a pre-registered human-in-the-loop validation gate.

5. **Research Integrity & Reproducibility:** Every dataset manifest, model checkpoint, evaluation JSON, and visualization is cryptographically locked with SHA-256 hashes, backed by a comprehensive **311-test automated test suite**.

---

2. Problem Statement & Domain Challenges

Conventional object detection models trained on standard benchmarks struggle in Indian urban environments due to several distinct domain challenges:

1. **Extreme Vehicle Heterogeneity (14 Classes):** Standard detectors classify cars, buses, and trucks. Indian traffic includes *Three-wheelers* (auto-rickshaws), *Two-wheelers* (scooters, motorcycles), *Light Commercial Vehicles (LCV)*, *Tempo-travellers*, *Mini-buses*, *Bicycles*, and

1. *Others* (handcarts, animal-drawn carts).

2. **Dense Occlusion & Non-Lane Behavior:** Vehicles cluster with negligible headway and lateral spacing. Two-wheelers weave between heavy vehicles, creating continuous boundary occlusions where vehicle contours blend.

3. **Drastic Scale Variations:** High-angle traffic cameras capture multi-axle trucks spanning 400×300 pixels in the foreground alongside distant motorcycles occupying fewer than 20×20 pixels in the background.

4. **Severe Class Imbalance:** Two-wheelers dominate roadway volume (~48% of validation annotations), whereas rare categories such as *Others* (31 instances) and *Mini-bus* (58 instances) represent less than 0.3% of the dataset.

5. **Adverse Environmental Conditions:** Tropical glare, high-contrast shadows, road dust, and non-standard vehicle modifications (luggage racks, tarpaulins) degrade feature extraction.

---

3. Data Engineering & Integrity Governance

3.1 Source Dataset: UVH-26

The project uses the **IISc AIM Urban Vehicle Heterogeneity (UVH-26)** dataset (Revision: `59f82c57821e8a54dc40bc1f42e83909dbad0b70`, licensed under CC BY 4.0).

• **Annotation Variant:** Majority Voting (MV) ground truth. STAPLE consensus annotations were explicitly excluded to preserve single-standard ground truth.

• **Image Geometry:** 1920×1080 high-definition RGB frames captured from elevated roadside infrastructure across Indian metropolitan intersections.

3.2 Full Catalogue Audit vs. Local Frozen Subset

Before training, an exhaustive audit evaluated the entire dataset catalogue against local disk assets:

Dataset Scope	Image Count	Bounding Box Count	Audit Verification Coverage
Full Source Catalogue	26,646	316,220	Schema syntax, coordinates, valid ranges, split balance
Frozen Train Subset	8,000	94,609	Image decoding, SHA-256 hashes, label synchronization
Frozen Validation Subset	2,000	24,342	Image decoding, zero train/val overlap, label

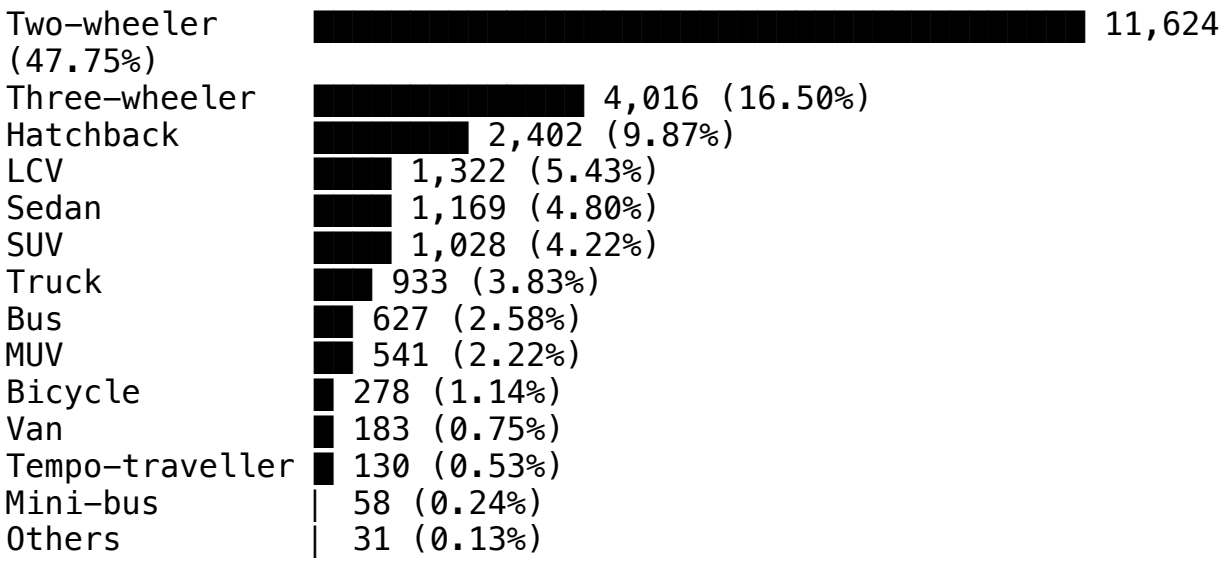
sanity			
Sealed Holdout Subset	1,500	~18,000	Preserved unopened for future unbiased generalization testing

3.3 14-Class Taxonomy & Class Imbalance

The 14 classes are mapped sequentially from UVH-26 Category IDs 1–14 to YOLO IDs 0–13:

```
[0: Hatchback]  [1: Sedan]   [2: SUV]           [3: MUV]
[4: Bus]
[5: Truck]      [6: Three-wheeler]              [7: Two-wheeler]
[8: LCV]
[9: Mini-bus]   [10: Tempo-traveller]           [11: Bicycle]
[12: Van]
[13: Others]
```

Class Distribution in Frozen Validation Subset (2,000 Images / 24,342 Objects)



3.4 Data Integrity, Quarantine & Manifest Security

To ensure reproducible experiments, the pipeline enforces strict data controls:

• **Corrupted Candidate Quarantine:** During initial subset preparation, three corrupted image files and two degenerate label records were quarantined. Replacements were selected deterministically to match class distributions without manual coordinate modification.

• **Frozen Manifest Version:** `uvh26_mv_yolo_v1/subsets/baseline_seed42_v2`

• **Cryptographic Manifest Hashes:**

• `manifest_combined.json`: `990054a93e300a90321db19b3d0bcd98a488a891cd4e2dbd88425f4eb592c2af`

• `manifest_train.json`:

• 8e4a72413caecc9defee68e75f498b71160b06fb33f30f6cbe2dc8b3677032cf

• manifest_val.json: fd23d2e417d70a8614b6312cb3eff1deb0ed98f0532269c22878269fdc5959d3

• class_mapping.yaml: 6fd0b458fed2cb174c924d4fa9ead86db0a292dc7c55fd0f4d4b3b9b20d08ea8

• **Zero Data Leakage:** SHA-256 image content hashing verified zero intersection between training, validation, and sealed subsets.

———————

4. Experimental Evolution: From Scratch to End

4.1 Phase 1 (E0): Compact One-Stage Baseline (YOLOv8n)

• **Objective:** Establish an audited, reproducible, lightweight detection baseline.

• **Architecture:** YOLOv8n (Anchor-free single-stage detector, 3,013,578 parameters).

• **Training Protocol:**

• Base weights: Official COCO-pretrained yolov8n.pt

• Data: Frozen 8,000 train / 2,000 validation images

• Hyperparameters: 30 epochs, image size 640×640, physical batch size 8, initial lr 0.000556, weight decay 0.0005, AdamW optimizer, warmup 3 epochs, patience 10.

• Compute: Apple Silicon MPS backend, AMP disabled, workers 0. Total training time: 16,029.86 s (~4.45 hours). Process exit 0.

• **Measured Performance (Frozen 2,000 Val):**

• **mAP@0.5:** 56.0049%

• **mAP@0.5:0.95:** 45.8407%

• **Precision (Macro):** 60.9993%

• **Recall (Macro):** 54.3547%

• **Harmonic Aggregate F1:** 57.4856% (Macro F1: 53.9397%)

• **Latency Benchmark (100-Image Synchronized MPS, Batch 1):**

• Median end-to-end: **31.73 ms** (31.89 still images/sec)

• Inference-only speed: **201.77 FPS** (4.96 ms/image)

———————

4.2 Phase 2 (E1): Model Capacity Scaling (YOLOv8s)

• **Objective:** Determine whether increasing backbone and neck capacity improves detection accuracy under identical training budgets.

• **Architecture:** YOLOv8s (11,141,018 parameters, a 3.61× increase in capacity; checkpoint

• size: 22.50 MB vs. 6.23 MB).

• **Training Protocol:** Identical 8,000/2,000 split, 30 epochs, seed 42, AdamW, batch 8, imgsz 640. Best checkpoint obtained at **Epoch 22**. Active training time: 28,459.26 s (~7.91 hours).

• **Comparative Results (E1 vs. E0 Baseline):**

Metric	E0 (YOLOv8n)	E1 (YOLOv8s)	Absolute Delta	Relative Gain
mAP@0.5:0.95	45.8407%	**52.4760%**	**+6.6353 pp**	**+14.47%**
mAP@0.5	56.0049%	**62.3274%**	**+6.3225 pp**	**+11.29%**
Harmonic Aggregate F1	57.4856%	**62.9658%**	**+5.4802 pp**	**+9.53%**
Macro Class F1	53.9397%	**59.1856%**	**+5.2459 pp**	**+9.73%**
Macro Precision	60.9993%	**65.0335%**	**+4.0342 pp**	**+6.61%**
Macro Recall	54.3547%	**61.0255%**	**+6.6708 pp**	**+12.27%**

• **Per-Class Improvements:** All 14 classes improved under E1. The largest gains occurred in underrepresented vehicles: *Van* (+13.81 pp), *Bicycle* (+12.22 pp), *Tempo-traveller* (+11.55 pp), and *Sedan* (+10.32 pp). *Three-wheeler* remained the highest performing class (78.14% AP50:95).

• **Engineering Recovery:** An evaluation export bug caused by validator hook incompatibilities was resolved via an atomic exporter without altering checkpoints or retraining. E0 reproduced identical baseline metrics.

• **Decision: E1 YOLOv8s was adopted as the Preferred Research Detector.**

4.3 Phase 2 (E2): Input Resolution Ablation Study (640 vs. 960)

• **Hypothesis:** Upscaling input resolution to 960×960 would significantly enhance feature representation for distant two-wheelers, bicycles, and occluded objects.

• **Experimental Design (Gate A):** Evaluate the frozen E1 Epoch 22 checkpoint at both 640×640 and 960×960 on the 2,000 validation images under identical hardware and software settings.

• **Pre-Registered Progression Criteria:**

• Advance to 960 training (Gate B) ONLY IF:

• Small-object AP50:95 improves by $\ge +2.0\text{ percentage points}$, OR

• Overall mAP50:95 improves by $\ge +1.0\text{ percentage point}$ with positive gains in key vehicle categories.

• **Measured Outcomes:**

Metric	E1 at 640	E1 at 960	Absolute Change	Gate Status
Overall mAP@0.5:0.95	52.4760%	52.5139%	**+0.0379 pp**	Failed ($\Delta < 1.0\text{ pp}$)
Small-Object Macro AP	27.3470%	20.2710%	**-7.0760 pp**	Failed ($\Delta < +2.0\text{ pp}$)
Medium-Object AP	31.8890%	37.0770%	+5.1880 pp	Informational
Large-Object AP	57.4230%	56.7650%	-0.6580 pp	Informational
Macro Precision	65.0335%	64.0242%	-1.0093 pp	Decreased
Macro Recall	61.0255%	61.0244%	-0.0011 pp	Negligible

• **Root Cause Analysis:** While raw two-wheeler small-object AP improved (+7.53 pp) and three-wheelers gained (+13.07 pp), small-object instances in rare classes suffered extreme variance (e.g., a single small SUV instance caused small SUV AP to drop from 60% to 0%). Furthermore, false-positive background detections increased across dense queues.

• **Scientific Decision: Gate A FAILED. Retain 640×640 resolution.** Gate B (30-epoch training at 960) was cancelled, preventing wasted compute.

4.4 Stages A–D (E3): Two-Stage Detector Exploration (Faster R-CNN ResNet-50-FPN)

• **Motivation:** Single-stage detectors struggle with severe occlusion and fine-grained classification. Two-stage detectors (Faster R-CNN) with a Region Proposal Network (RPN) offer alternative proposal mechanics.

• **Stage A & B (Pilots):** Evaluated official PyTorch torchvision Faster R-CNN (ResNet-50-FPN, 41.37M parameters). Explored inverse-square-root class-frequency weighted sampling vs. uniform sampling across 1,000 train / 250 calibration images. Uniform sampling provided greater optimization stability.

• **Stage C0 (Fault Tolerance):** Built an enterprise-grade detached supervisor with atomic epoch-boundary recovery, RNG/optimizer restoration, and execution locks.

• **Stage D (Full 20-Epoch Run):** Trained Faster R-CNN for 20 epochs on 8,000 images using Apple MPS. Checkpoint selection identified **Epoch 13** as optimal.

• **Stage D Standalone Evaluation (Calibration 500 Images):**

• **AP@0.5:** 56.3561%

• **AP@0.5:0.95:** 42.9070%

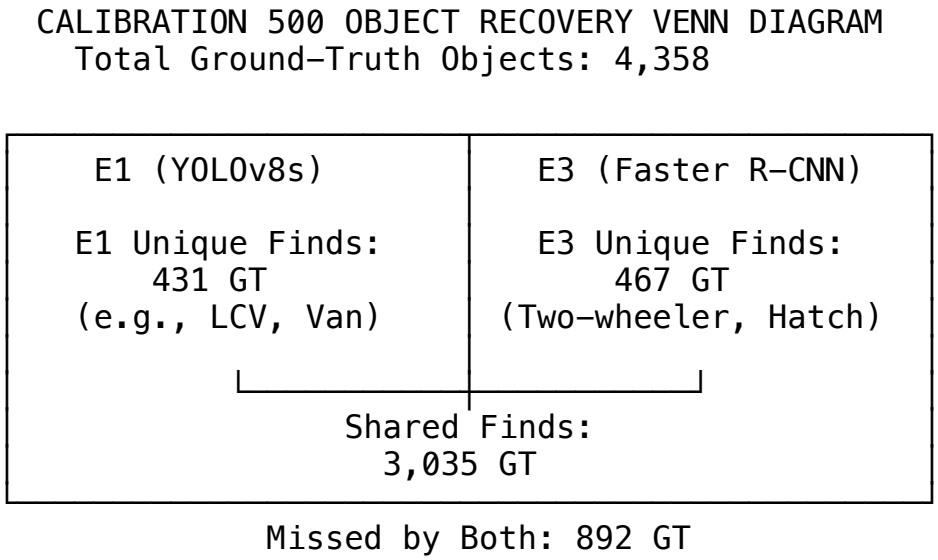
• **Harmonic Aggregate F1:** 53.0884% (Macro F1: 52.63%)

• **Latency (MPS Batch 1):** Median **112.90 ms** (8.75 still images/sec) — nearly 4× slower than YOLOv8s.

4.5 Stage E: Matched Calibration Evaluation & Complementarity Analysis

To ensure fair comparison, E1 (YOLOv8s) and E3 (Faster R-CNN) were evaluated on the identical **calibration500** subset using a unified COCO-style evaluator and standardized class mappings. Operating thresholds were calibrated over a 100-point grid to maximize macro class F1:

• **Calibrated Operating Thresholds:** E1 = **0.34**, E3 = **0.43**.



• **Key Insight:** E1 clearly led in aggregate metrics (48.28% vs. 42.91% AP50:95). However, E3 recovered **467 ground-truth objects completely missed by E1** at their operating points (including 177 two-wheelers and 102 hatchbacks). This confirmed significant architectural complementarity, warranting ensembling exploration.

4.6 Stage F (E4): Model Ensembling & Fusion Feasibility Study

• **Hypothesis:** Combining YOLOv8s and Faster R-CNN predictions via box fusion would harness proposal complementarity to achieve higher detection accuracy.

• **Experimental Protocol:** Evaluated 19 fusion configurations across 9 confidence thresholds using a strictly bounded **5-fold cross-validation scheme on the calibration500 set**.

• **Evaluated Fusion Families:**

1. *Simple Union:* Concatenation followed by Non-Maximum Suppression (NMS).

2. *Weighted Boxes Fusion (WBF):* Soft box clustering and confidence-weighted coordinate averaging.

3. *Recovery Mode:* Retaining high-confidence primary E1 boxes and recovering secondary E3 boxes only in non-overlapping regions.

• **Pre-Registered Acceptance Criteria:**

• Fusion adopted ONLY IF:

• Out-of-fold mAP@0.5:0.95 gain $\ge +1.00\text{ percentage point}$, AND

• Out-of-fold Macro F1 gain $\ge +2.00\text{ percentage points}$.

• **Out-of-Fold 5-Fold Cross-Validation Results:**

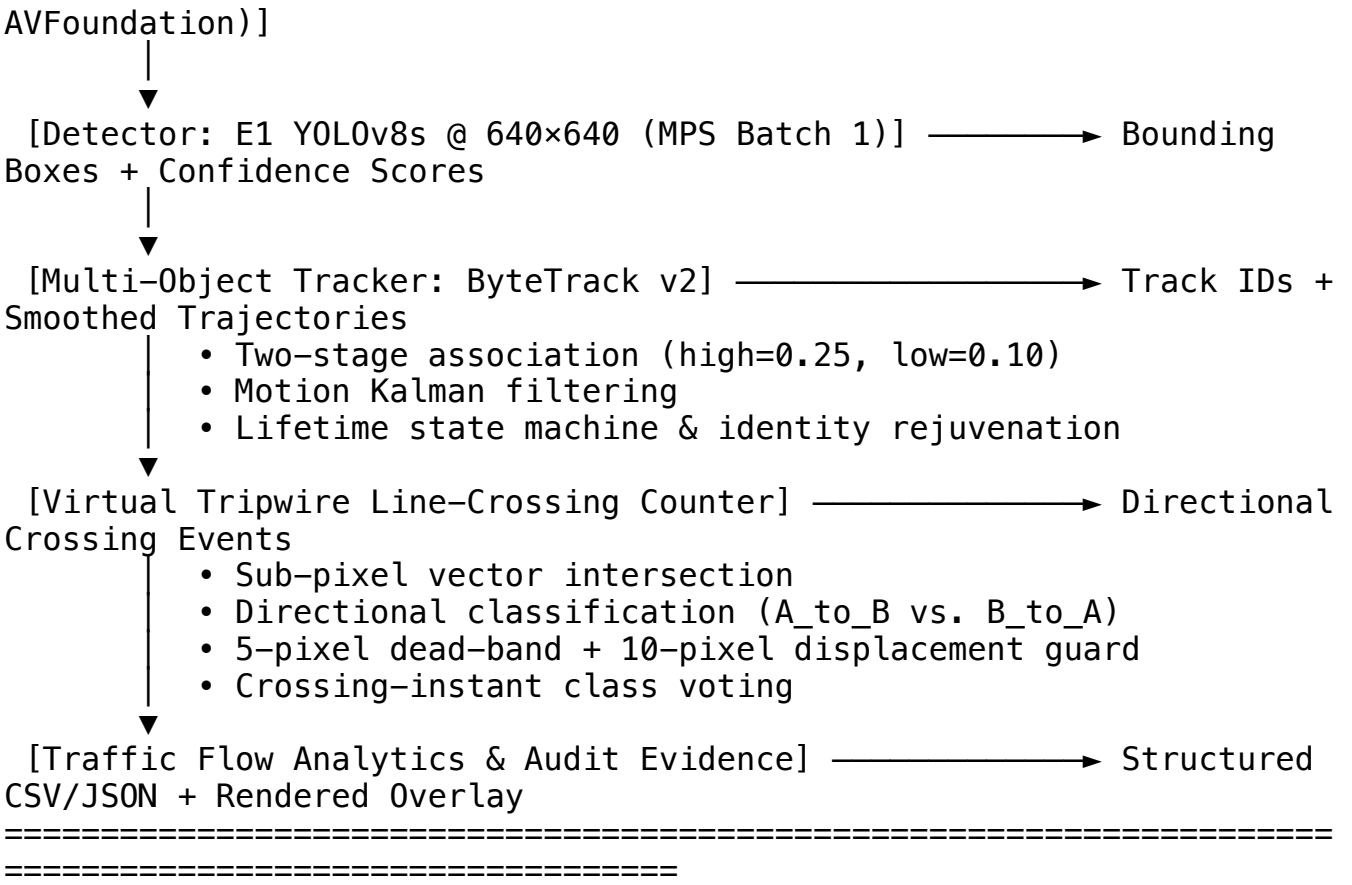
Family	Precision	Recall	Harmonic F1	Macro F1	AP50	AP50:95	Δ AP50:95
Baseline (E1 Only)	62.04%	56.77%	59.29%	58.62%	57.38%	48.28%	Reference
Union	56.76%	59.92%	58.29%	57.74%	60.20%	46.60%	-1.68 pp
Recovery	59.87%	57.50%	58.66%	57.88%	55.88%	46.72%	-1.56 pp
WBF (Best Fusion)	**62.92%**	**57.63%**	**60.16%**	**59.47%**	**58.88%**	**49.24%**	**+0.96 pp**

• **Sequential Latency Overhead:** Running E1 + E3 + WBF sequentially required **156.78 ms mean latency** (6.38 images/sec), compared to 30.29 ms for E1 alone.

• **Scientific Decision: FUSION REJECTED.** The observed WBF gain (+0.9603 pp AP, +0.8530 pp Macro F1) failed to meet the pre-registered thresholds (+1.0 pp AP, +2.0 pp F1), while introducing unacceptable latency. **E1 YOLOv8s was retained as the sole production model.**

4.7 Phase 3A & 3B: Video Traffic Analytics, Tracking & Line-Crossing Counting

```
================================================================================
                                VIDEO TRAFFIC ANALYTICS PIPELINE
================================================================================
 Raw Video Stream (1080p @ 30 FPS)
        │
        ▼
 [Hardware Frame Decoder (OpenCV /
```



• **Objective:** Extend static frame detection to video tracking and directional traffic counting.

• **Test Footage:** Permitted 900-frame (30.0 s, 1920×1080 @ 30 FPS) video sequence from a busy Bangalore urban corridor (Wikimedia Commons CC BY-SA 4.0; SHA-256: `42833cd01527ee03c6bee7b9c6caba5baa75532c57ca3262757dd51f0347dab2`).

• **Core Tracking & Counting Innovations:**

• *ByteTrack Adaptation:* Retained low-confidence detection associations (down to 0.10) to maintain track continuity across partial occlusions.

• *Tripwire Logic:* Virtual counting line defined between $(153.6, 648)$ and $(1382.4, 648)$. Enforced a 5-pixel dead-band, 10-pixel minimum displacement, and single-event-per-track constraints to prevent double-counting of stationary or reversing vehicles.

• *Class Voting:* Frozen confidence-weighted class majority voting over the 5 frames preceding the tripwire intersection.

• **Pipeline Processing Speed Benchmarks (900 Frames, Apple M5 MPS):**

Pipeline Mode	Processing Speed	Median Frame Time	95th Percentile Time	Total Wall Time	Real-Time Factor
Detection Only	85.69 FPS	11.47 ms	13.00 ms	10.50	

s	2.86×				
Tracking Only	85.43 FPS	11.13 ms	12.92 ms	10.54 s	2.85×
Counting Mode	83.43 FPS	11.32 ms	13.58 ms	10.79 s	2.78×
Complete End-to-End	**60.04 FPS**	**16.41 ms**	**19.19 ms**	**14.99 s**	**2.00×**

• **Predicted Traffic Volume:** On the 30-second benchmark sequence, the system recorded **40 vehicle crossings** (12 downward `A_to_B`, 28 upward `B_to_A`) across 10,194 detected instances, maintaining a maximum of 22 concurrently tracked objects.

• **Human-in-the-Loop Ground-Truth Review Gate:** In adherence to strict scientific standards, automated predictions are explicitly separated from verified ground-truth counts. A web-based frame-by-frame ground-truth annotation interface was built to enable independent human verification before claiming definitive counting accuracy.

---

5. Master Performance & Latency Matrix

The following table consolidates all completed experiments in this research program. Metrics reflect their pre-registered evaluation sets:

Model ID	Architecture	Evaluation Split	Parameters	Precision	Recall	Harmonic F1	Macro F1	mAP@0.5	mAP
E0	YOLOv8n (640)	Frozen Val 2,000	3.01 M	61.00%	54.35%	57.49%	53.94%	56.00%	45.8
E1 (Best)	**YOLOv8s (640)**	**Frozen Val 2,000**	**11.14 M**	**65.03%**	**61.03%**	**62.97%**	**59.19%**	**62.33%**	**52.4**
E2	YOLOv8s (960)	Frozen Val 2,000	11.14 M	64.02%	61.02%	62.49%	58.77%	62.64%	52.5
E1 (Calib)	YOLOv8s (640)	Calibration 500	11.14 M	61.48%	58.17%	59.78%	59.10%	57.38%	48.2
E3	Faster R-CNN	Calibration 500	41.37 M	53.63%	56.88%	55.20%	54.65%	56.36%	42.9
E4 (WBF)	Ensemble (E1+E3)	Calibration 5-Fold	52.51 M	62.92%	57.63%	60.16%	59.47%	58.88%	49.2

Note: Models evaluated on Calibration 500 should not be ranked directly against Validation

2,000 runs due to subset size differences.

---

6. System Architecture & Codebase Layout

The repository is structured following modular software engineering practices:

```
Deep Learning/
├── app.py                                  # Main Streamlit faculty
review application
├── dashboard/                              # Dedicated multi-page
dashboard service
│   ├── app.py                              # Secondary dashboard entry
point
│   ├── pages.py                            # High-level page routing and
UI layout
│   ├── video_analytics.py                  # Video counting & interactive
tripwire UI
│   ├── presentation.py                     # Presentation mode controller
│   └── services.py                         # Artifact loading and caching
backend
├── components/                             # Modular Streamlit components
│   ├── hero.py                             # Banner and executive KPI
summary
│   ├── metrics.py                          # Performance tables and
metric cards
│   ├── dataset.py                          # Dataset audit & class
distribution
│   ├── eda.py                              # Bounding box & aspect ratio
analysis
│   ├── baseline.py                         # E0 baseline vs E1 vs E2
comparisons
│   ├── e3.py                               # Faster R-CNN training &
evaluation
│   ├── fusion.py                           # Stage F ensemble & fusion
analytics
│   ├── testing.py                          # Synthetic tests & regression
logs
│   ├── pipeline.py                         # Video pipeline interactive
diagrams
│   └── status.py                           # Milestone tracking &
research roadmap
├── src/                                    # Core Python source library
│   ├── data/                               # Ingestion, validation, and
conversion
│   │   ├── audit_baseline_subset.py        # 10,000-image dataset
integrity audit
│   │   ├── convert_to_yolo.py              # UVH-26 COCO to YOLO format
```

```
converter
|   |   ├── prepare_subset.py          # Deterministic seed-42 subset
split
|   |   └── validate_raw.py            # Coordinate sanity & boundary
checks
|   ├── training/                      # Training scripts & fault
tolerance
|   |   ├── train_baseline.py          # YOLOv8n baseline training
pipeline
|   |   ├── train_e1.py                # YOLOv8s capacity scaling
pipeline
|   |   └── epoch_resume.py            # Detached supervisor &
checkpoint recovery
|   ├── evaluation/                    # Standalone evaluators &
metrics
|   |   ├── evaluate_baseline.py       # YOLOv8 validation & PR curve
exporter
|   |   ├── evaluate_e2.py             # Resolution study evaluation
harness
|   |   ├── e2_size_ap.py              # Scale-stratified COCOeval
adapter
|   |   ├── benchmark_e2.py            # Synchronized MPS latency
benchmarking
|   |   ├── comparison_protocol.py     # Multi-model evaluation
protocols
|   |   ├── complementarity.py         # Object-level recovery Venn
analytics
|   |   └── fusion.py                  # WBF and NMS box ensembling
engines
|   ├── experiments/                   # Faster R-CNN & exploratory
models
|   |   ├── train_frcnn.py             # PyTorch Faster R-CNN
training loop
|   |   └── stage_b.py                 # Weighted vs unweighted pilot
studies
|   └── video/                         # Real-time traffic analytics
|       ├── tracking.py                # ByteTrack multi-object
tracker
|       ├── counting.py                # Virtual tripwire counting
logic
|       ├── phase3b.py                 # End-to-end video
benchmarking pipeline
|       ├── phase3b_annotations.py     # Ground-truth annotation
manager
|       └── phase3b_evaluation.py      # Event-level matching and
metrics
├── configs/                           # Immutable configuration
YAMLs
|   ├── baseline_yolov8n.yaml          # E0 hyperparameters and data
```

```
paths
│   ├── E1_yolov8s_uvh26_mv_640_seed42.yaml
│   ├── class_mapping.yaml             # 14-class mapping and color
codes
│   ├── phase3_bytetrack.yaml          # ByteTrack association
thresholds
│   └── phase3_counting.yaml           # Line coordinates and dead-
bands
├── reports/                           # Generated evaluation
artifacts
│   ├── audit/                         # Integrity audits and
delivery proofs
│   ├── comparisons/                   # Cross-model delta tables and
CSVs
│   ├── evaluations/                   # Full JSON evaluation bundles
│   └── phase3b/                       # Video benchmark metrics &
event logs
├── tests/                             # Comprehensive automated test
suite
│   ├── test_annotation_conversion.py  # Coordinate conversion
correctness
│   ├── test_e1_comparison.py          # E1 evaluation integrity
checks
│   ├── test_e2_resolution.py          # Resolution study regression
tests
│   ├── test_epoch_resume.py           # Checkpoint recovery unit
tests
│   ├── test_fusion.py                 # Box fusion logic
verification
│   ├── test_phase3b.py                # Video tracking and counting
tests
│   └── ... (29 test modules)
└── docs/                              # Formal research reports and
protocols
    ├── ALL_MODEL_PERFORMANCE.md       # Consolidated metrics record
    ├── phase_reports/                 # Detailed phase documentation
    └── faculty_review/                # Executive summaries &
scripts
```

---

7. Streamlit Interactive Research & Faculty Dashboard

The project includes an interactive **Streamlit Dashboard** (`app.py` and `dashboard/app.py`) designed for peer review, demonstration, and error analysis.

Key Capabilities

1. **Saved Evidence Mode (Zero-GPU Requirement):** The entire application operates directly from pre-computed, hash-verified JSON, CSV, and PNG artifacts. Reviewers can explore full

1. metrics, confusion matrices, and latency charts on any laptop without needing PyTorch or GPU hardware.

2. **Interactive Model Zoo & Comparison Matrix:** Direct side-by-side metric comparisons between E0, E1, E2, E3, and E4 with interactive confidence threshold sliders and class filtering.

3. **Exploratory Data Analysis (EDA) Explorer:** Visualizations of aspect-ratio distributions, bounding-box scale distributions, and category co-occurrence heatmaps across the 24,342 validation objects.

4. **Error Analysis & Prediction Inspector:** Inspection of false positives, false negatives, and classification confusions across dense traffic scenes, highlighting common failure modes (e.g., *Tempo-traveller* vs. *LCV*).

5. **Video Traffic Analytics Suite:** Interactive video playback with tracked bounding-box overlays, directional counting line visualizers, trajectory trails, and instantaneous vehicle velocity indicators.

6. **In-Browser Human Ground-Truth Annotation Tool:** A frame-by-frame annotation tool allowing reviewers to seek through video sequences, log crossing timestamps, assign vehicle classes, and compare human ground truth directly against automated predictions.

7. **New Data Evaluation Ingestion:** Upload new YOLO-formatted datasets or raw video clips to run automated evaluation with bootstrap confidence intervals.

---

8. Software Engineering Rigor & Reproducibility Guarantees

This project adheres to strict scientific and software engineering principles:

• **Comprehensive Automated Testing:** Over **311 passing unit and integration tests** verifying data adapters, coordinate conversions, metric computations, and recovery mechanisms (`python –m pytest`).

• **Fail-Closed Security & Atomic I/O:** All checkpoint exports and metric summaries are written to temporary staging files and published via atomic directory renames. If an evaluation run fails or produces non-finite values (NaN/Inf), the pipeline halts immediately without polluting benchmark registries.

• **Strict Cryptographic Hashing:** Checkpoints, configuration files, split manifests, and evaluation bundles maintain recorded SHA-256 signatures in `experiment_registry.json`. Any unauthorized modification invalidates the evaluation.

• **Sealed Evaluation Holds:** The 1,500-image holdout set remains cryptographically sealed to prevent validation overfitting.

• **Hardware-Synchronized Benchmarking:** Latency benchmarks use explicit device synchronization (`torch.mps.synchronize()`), eliminate the first 10 warm-up runs, and record per-stage breakdowns (preprocess, inference, postprocess, NMS).

---

9. Step-by-Step Execution & Reproduction Guide

9.1 Environment Setup

```
# 1. Clone the repository
git clone https://github.com/manavshah409/Deep_Learning_UHV.git
cd "Deep Learning"

# 2. Create and activate a Python 3.12 virtual environment
python3.12 -m venv .venv
source .venv/bin/activate

# 3. Install core dependencies
pip install --upgrade pip
pip install -r requirements.txt

# 4. Install dashboard and evaluation extensions
pip install -r requirements-dashboard.txt
pip install -r requirements-e2-evaluation.txt
```

9.2 Running Automated Tests

```
# Run the full test suite (311 tests)
python -m pytest -q

# Run specific functional test suites
python -m pytest tests/test_e1_comparison.py
python -m pytest tests/test_phase3b.py
python -m pytest tests/test_fusion.py
```

9.3 Launching the Interactive Dashboards

```
# Launch the primary Phase 1 & Research Review Application
streamlit run app.py

# Launch the Video Traffic Analytics & Multi-Page Faculty Dashboard
streamlit run dashboard/app.py
```

9.4 Running Offline Verification & Benchmarking

```
# Validate data and artifact integrity
python scripts/validate_e2_artifacts.py

# Benchmark E1 inference latency on local hardware
python -m src.evaluation.benchmark_inference --config
configs/E1_yolov8s_uvh26_mv_640_seed42.yaml

# Run the complete Phase 3B video tracking & counting pipeline
python -m src.video.phase3b --config configs/phase3a_bangalore01.yaml
```

---

10. Failure Analysis, Limitations & Future Roadmap

10.1 Observed Failure Modes

1. **Extreme Dense-Queue Occlusion:** In congested intersection queues where bumper-to-bumper distance drops below 0.5 meters, two-wheelers positioned directly behind large commercial trucks are frequently occluded and missed.

2. **Fine-Grained Classification Ambiguities:** Visual similarity between *Tempo-travellers* and commercial *Vans* or *LCVs* causes occasional class swapping across consecutive video frames.

3. **Rare Class Support:** The *Others* class (31 validation instances) achieved zero recall at default operating thresholds. Rare categories require targeted data augmentation.

4. **Handheld Camera Shake:** Severe video shake or low-angle glare affects virtual tripwire intersections if the baseline camera geometry shifts relative to the road plane.

10.2 Future Roadmap

• [ ] **Targeted Minority Fine-Tuning:** Collect and curate additional samples of rare categories (*Mini-bus*, *Others*, *Bicycle*) and execute controlled fine-tuning with class-balanced focal losses.

• [ ] **Temporal Multi-Frame Smoothing:** Integrate lightweight temporal feature aggregation across adjacent frames to stabilize classification in dense traffic queues.

• [ ] **Road Plane Homography & 3D Bounding Boxes:** Calibrate camera geometry to project pixel trajectories onto metric road coordinates, enabling accurate vehicle speed estimation (km/h) and lane occupancy analytics.

• [ ] **Edge Deployment:** Export the optimized E1 YOLOv8s model to Apple CoreML (with 16-bit floating-point quantization) and NVIDIA TensorRT for deployment on embedded edge devices (Jetson Orin).

---

11. Project Metadata & Sign-off

• **Author & Researcher:** Manav Shah

• **Academic Context:** Deep Learning for Intelligent Transportation Systems (ITS)

• **Status:** Phase 1, Phase 2 (E1, E2), Stages A–F (E3, E4), and Phase 3B Completed & Audited

• **Preferred Model Checkpoint:**
`runs/E1_yolov8s_uvh26_mv_640_seed42/weights/best.pt`

• **Documentation Version:** 1.0.0 (Master Release)